
TECHNICAL OVERVIEW

Technical Overview

Architecture, control boundaries, and failure behavior

Aurelius

AI infrastructure orchestration and cost optimization

Document version 1.0 · May 2026

Purpose

This overview describes the engineering structure of Aurelius for platform and infrastructure architecture review: the components, the data flow between them, the boundary of what the system controls, and its behavior under failure.

Aurelius is a decision pipeline. Market and workload data enter through ingestion; a forecaster produces calibrated price expectations; an optimization engine selects placement and timing under constraints; a safety gate vets each decision; and the result is either reported (replay and shadow modes) or handed to an execution adapter (controlled execution). Every stage is deterministic given its inputs and a fixed seed, and every stage degrades to the customer's baseline behavior rather than to an unsafe state.

FIGURE

Ingestion » Forecasting » Optimization » Safety gate

optional signals: carbon · queue · GPU health

Reporting (replay, shadow) Execution adapters (dry-run default)

Component and data flow.

Components

Data ingestion

Ingestion normalizes every external signal into a canonical, timestamped, per-region schema. Wholesale prices come from direct ISO and TSO connectors returning hourly day-ahead and real-time settlement prices. A provenance layer tags every record with its source and market type and enforces an admissibility gate, so that sandbox or synthetic data is structurally barred from any savings path. Workload traces are ingested from a customer export; a minimal set of fields is sufficient to begin, with per-class defaults applied for anything omitted. Optional signals — carbon intensity, queue state, and GPU telemetry — enter through the same schema and are joined to price data by region and hour.

Forecasting

The forecaster produces calibrated quantile predictions of future prices per region and horizon. The validated production model is a gradient-boosted quantile model with lagged price, seasonality, and volatility-regime features for spike detection. Calibration of the prediction interval is treated as a first-class metric alongside point accuracy, because the safety gate depends on the interval being trustworthy. A deterministic baseline forecaster is retained as a fallback and as a benchmark reference.

Optimization engine

The optimizer assigns each job a region and start window to minimize a weighted objective combining energy cost with optional terms for carbon, forecast risk, SLA penalty, queue delay, GPU health, and data transfer. It does so subject to hard constraints: each job's earliest-start and deadline window, its allowed and forbidden regions, per-region capacity, and minimum power level. The risk term penalizes scheduling into high-uncertainty periods, so a wide forecast interval naturally biases the optimizer toward the safer, baseline-like choice. Optional signal weights default to zero, so a deployment supplying only prices behaves as a price-only optimizer; other signals activate only when both data and a cost weight are provided.

Policy and safety gate

Before any decision is emitted, the safety gate evaluates the forecast interval against a workload-specific downside threshold — most conservative for real-time inference, most permissive for training. If the projected downside exceeds the threshold, or if no valid interval is available, the gate blocks the optimized decision and the system falls back to the current-price baseline. The gate is fail-closed: absence of evidence defers to the baseline rather than proceeding. Hard constraints are enforced by the optimizer and re-checked at the gate; a decision that would violate them is never produced.

Shadow mode

Shadow mode is the mechanism for customer-specific validation. A single-pass runner makes the decisions the optimizer would make live, training only on price data preceding the decision time, and records one decision per job: the chosen region and start, the forecast, the predicted cost, and the baseline cost. Real-time settlement prices are never visible at decision time. After the settlement window closes, a separate step fills in the realized cost from settlement data, and a report compares predicted to realized savings. This separation — decide, then realize from independent data — is what makes the result credible rather than self-graded.

Scheduler integration

Execution adapters translate decisions into actions for common schedulers, plus a replay interface. All adapters default to dry-run, log every attempted action for audit, support a global kill switch, and require a signed policy bundle to enter live mode. These adapters are unit-tested against mocks; they have not yet been validated against live production infrastructure, and resource-mapping heuristics are deployment-specific and should be reviewed before controlled execution.

Control boundary

Aurelius reads market data and a workload description, and writes decisions. In replay and shadow modes it writes only to its own report files and never contacts the customer's scheduler. In controlled execution it submits to the scheduler through an adapter, but only for workloads explicitly in scope and only under an active policy. The system never holds workload data or model weights, and the only credentials it needs are read-only market-data keys plus, for controlled execution, whatever the chosen adapter requires.

The boundary is deliberate: Aurelius owns the decision, the customer's platform owns the execution. This keeps the blast radius of any fault contained to a suboptimal-but-valid placement, never an unsafe action.

Failure modes and fallback

Every failure resolves to the customer's existing behavior or a strictly safe subset of it. There is no path in which Aurelius produces a placement worse than the baseline it is measured against, because the baseline is always available as the fallback.

Condition	Behavior
Forecast unavailable or invalid	Safety gate blocks; fall back to current-price baseline
Forecast interval too wide / high downside	Optimized decision blocked; baseline used
Market feed missing for a region	Region excluded from routing; remaining regions used
Optional signal absent	Corresponding objective term is zero; price-only behavior
Hard constraint unsatisfiable	Job placed at constrained baseline; flagged in report
Execution adapter error (live mode)	Logged; kill switch available; no retry into unsafe state

Reproducibility

Forecasting and optimization are deterministic under a fixed random seed. Benchmarks pin the data window, baseline set, fold structure, and seed, and archive their outputs, so any reported figure can be regenerated. The validation methodology and its controls are documented in the Benchmark Appendix.